

Lab 2: Algorithm Complexity Analysis

Total Points: 50

Objectives

- Analyze the **time complexity** of algorithms using Big-O notation.
- Distinguish between **constant, linear, logarithmic, and quadratic** growth rates.
- Identify the **dominant term** in a complexity function.
- Compare algorithms based on **growth rate rather than execution time**.
- Practicing in **recursive thinking**.
- Writing **recursive function**.

Part 1 – [15 points] Manual Complexity Analysis (answers.pdf)

For each of the following code fragments:

1. Clearly identify the **basic operation**.
2. Count how many times it executes as a function of n.
3. Write the **Big-O time complexity**.

Question 1 – [2 point]

```
for (int i = 0; i < n; i++) {  
    cout << i << endl;  
}
```

Question 2 – [3 points]

```
for (int i = 0; i < n; i+=2) {  
    for (int j = 0; j < n; j++) {  
        cout << i + j << endl;  
    }  
}
```

Question 3 – [3 points]

```
for (int i = 0; i <= n; i++) {  
    for (int j = 1; j <= i; j++) {  
        k++;  
    }  
}
```

Question 4 – [4 points]

```
for (int i = 1; i < n; i*=2) {  
    for (int j = 0; j < 20; j++) {  
        k += i + j;  
    }  
}
```

Question 5 – [3 points]

Given the function:

$$f(n) = 10n\log_2 n + 5n^2 + 100$$

- Identify the **dominant term**
- Write the **Big-O notation**
- Briefly justify your answer

Part 2 – Algorithm Comparison (answers.pdf)

Question 6 - [5 points] You are given the following two algorithms with their time complexity functions:

- Algorithm A: $f(n) = 5n^2 + 3n + 20$
- Algorithm B: $g(n) = 100\log n + n$

Answer the following:

1. Which algorithm is **asymptotically more efficient**?
2. For small values of n , which algorithm *might* run faster in practice?
3. Explain **why algorithm comparison is not based on execution time alone**, as discussed in the lecture.

Part 3 – Recursion

Question 7 (programming.cpp)

- A. **[3 points]** Write a function called **decimal** that takes one hexadecimal character (such as 0–9 or A–F) and returns its decimal value.
- B. **[2 points]** Find a formula that converts a hexadecimal string with n characters into its decimal value. **Put your formula as a comment in the programming.cpp**

Assume you already know:

- the decimal value of the first character, and
- the decimal value of the rest of the string (everything except the first character).

For example, if the string is "A12F":

- the decimal value of 'A' is 10, and
- the decimal value of "12F" is 303.

Using this information, write a formula that gives the decimal value of "A12F".

- C. **[5 points]** Use parts A and B to create a recursive solution:

- Use the function from part A as the base case
- Use the formula from part B as the recursive step

Then, write a recursive function that takes a hexadecimal string as input and returns its decimal value.

Question 8 – [5 points] What does the following functions compute? (answers.pdf)

```
int mystery (int n)
{
    if (n < 1)
        return 0;
    else if (n % 2 == 0)
        return mystery(n-1);
    else return 1 + mystery(n-1);
}
```

Question 9 – [5 points] What does the following functions compute?

```
boolean mystery(int[] x, int b, int c) {  
    if (b > 0)  
        if (x[b-1] == c)  
            return true;  
        else  
            return mystery(x, b-1, c);  
    return false;  
}
```

Question 10 – [5 points]

```
int mystery(int[] x, int b, int c)  
{  
    if (b > 0)  
        if (x[b-1] == c)  
            return b-1;  
        else  
            return mystery(x, b-1, c);  
    return -1;  
}
```

Part 4 – Reflection (answers.pdf)

Question 11 - [5 points] Why does Big-O notation:

- Ignore constants?
- Ignore non-dominant terms?

Explain using an example from this lab or the lecture.

Submission

Submit a zip file including **programming.cpp** and **answers.pdf** in the Brightspace before the due date.